

Phase 1 Checkpoint Quiz — Data & Storage (objective)

Purpose. Lock in the *transferable engineering principles* from Phase 1 before we build `src/`.
Not a finance-terminology test — every question targets an AI/ML, MLOps, security, or software-architecture idea that the finance data merely made concrete.

How to take it. Each question is marked **(select one)** or **(select all that apply)**. Pick the letters, then expand `<details>` to check — the answer names the principle underneath. No writing required; just decide and verify.

A. The audit model & data reality

Q1 (select all that apply). In Aegis's "*an audit is a diff*" framing, which are true?

- A. The claim side (the filing) can be real, public data.
- B. The evidence side (the transaction ledger) is public for public companies.
- C. The evidence ledger is confidential, so we simulate it.
- D. Reconciliation compares the claimed aggregate against the summed evidence.

1. Answer

A, C, D. (B is false — the granular ledger is private/MNPI; only the summary is public.)

Q2 (select all that apply). Why anchor the synthetic ledger's total to a *real* reported figure?
Principle: design around what data can *legitimately* be real; don't pretend a private source is available.

- A. It models a correctly-kept book — a labeled *negative control*.
- B. Purely for cosmetic realism.
- C. Because we set the total (and any delta), we own the ground truth for evaluation.
- D. So reconciliation always returns MATCH.

2. Answer

A, C. (D is only true for the clean case and isn't the reason; B undersells it.) **Principle:**

synthetic data is most useful when it's *labeled by construction* — you know the right answer.

B. Ingestion principle

Q3 (select all that apply). Why pull the figure as *structured* data instead of parsing the document text?

- A. It keeps a non-deterministic parser/LLM out of the exact-number path.
- B. It's deterministic, cheap, and reproducible.
- C. It's the only technical way to obtain the number.

- D. It captures the qualitative narrative better than text.

3. Answer

A, B. (C false — you *could* parse it; D false — structured data *drops* the narrative.) **Principle:** Q4 (select one). What do we deliberately give up by going structured-first? Keep non-determinism out of the exact-value path.

- A. Numerical exactness.
- B. The qualitative grounding quote / narrative (deferred to the retrieval module).
- C. Reproducibility.
- D. Provenance of the figure.

4. Answer

B. Numbers from structure; quotes from documents (added back later, schema-constrained + cited).

C. Correctness & determinism

Q5 (select one). Why is money integer-cents / Decimal , never float ?

- A. Floats are slower to add.
- B. Float rounding accumulates, so a *clean* book shows a phantom discrepancy — a false positive from your own number type.
- C. Decimal uses less memory.
- D. Databases can't store floats.

5. Answer

B. Principle: pick the representation that makes the invariant *exact*, not "close enough."

Q6 (select all that apply). Seeding the generator so re-runs are byte-identical gives which property, and why does an *auditor* need it?

- A. Reproducibility / determinism.
- B. A finding can be re-derived from recorded inputs (trust / defensibility).
- C. It improves model accuracy.
- D. It's required for float math to work.

6. Answer

A, B. Principle: reproducibility is a first-class MLOps requirement — seed and record

Q7 (select one). Your *clean* test ledger shows a 3-cent discrepancy. Most likely culprit? Everything that feeds a result.

- A. The reconciliation logic is broken.
- B. The data generator — the "clean" fixture isn't exactly clean, so your negative control is lying.
- C. Database rounding.
- D. The source reported a wrong number.

7. Answer

B. Principle: if your fixtures/labels aren't exactly correct, your metrics are garbage — validate what you measure against before trusting it.

D. Evaluation thinking (MLOps)

Q8 (select all that apply). Why need **both** a clean and a discrepant ledger to trust a detector?

- A. Clean = labeled negative (should stay silent).
- B. Discrepant = labeled positive (should fire).
- C. A discrepant-only test can't catch a detector that false-positives on clean books.
- D. It doubles the training-set size.

8. Answer

A, B, C. Principle: a meaningful metric needs both classes (precision *and* recall); the negative case is what exposes a "cries wolf" detector.
Q9 (select all that apply). Why is "owning the ground truth by construction" valuable for evaluating the **LLM** parts later?

- A. You can grade the downstream agent because you know the correct delta.
- B. It removes the need for the LLM entirely.
- C. It lets you measure a probabilistic system against a known answer.
- D. It guarantees the LLM will be correct.

9. Answer

A, C. Principle: design data so the output is *checkable against a known truth* — that's what makes rigorous evaluation of a probabilistic system possible.

E. Security — the LLM as untrusted input

Q10 (select one). Why does parameterized SQL matter *more* in Aegis than in a typical app?

- A. Aegis handles more traffic.
- B. An LLM/agent authors the query, so the query string is untrusted input crossing a security boundary.
- C. Postgres is inherently less secure.
- D. Simply because the data is financial.

0. Answer

B. (D is tempting, but the precise reason is the *LLM-authored* query, not the data's topic.)

Q11 (select all that apply). Which controls keep "*the LLM chose the query*" from becoming "*the LLM ran arbitrary SQL*"?
Principle: treat any model-generated string crossing a boundary as hostile until proven safe.

- A. Parameterized queries (bind, never interpolate).
- B. A read-only database role.
- C. Query allow-listing / templates.
- D. Trusting the model because it's usually right.

1. Answer

A, B, C. Defense in depth at the tool (MCP) boundary.

F. The SourceConnector contract (architecture)

Q12 (select all that apply). In the connector contract, which statements are true?

- A. The `IngestionResult` envelope is uniform across all sources.
- B. The `records` payload schema is keyed by `kind`, not by source.
- C. Consumers should branch on the *source name* to know the schema.
- D. "Agnostic" means there is no schema at all.

2. Answer

A, B. (Consumers branch on `kind`, not source; agnostic $\neq$ schemaless.) **Principle:** normalize onto a small closed set of canonical shapes; depend on the contract, not the origin. *"an adapter, not a pipeline"*

- A. The connector conforms to the canonical envelope (its `normalize` is an anti-corruption layer).
- B. A registry + dispatcher routes by source, so `ingest` never changes.
- C. Each new source rewrites the `ingest` function.
- D. An LLM auto-generates a fresh pipeline per source.

3. Answer

A, B. **Principle:** conform-to-contract + registry dispatch = additive extensibility.

Q14 (select all that apply). When is `normalize` plain deterministic code vs. when does an LLM legitimately enter *at runtime*?

- A. Structured source $\rightarrow$ deterministic field-map, no LLM in the number path.
- B. Unstructured (value buried in prose) $\rightarrow$ LLM extraction into a fixed schema, with a citation.
- C. Every structured field should be reformatted by an LLM for safety.
- D. Design-time LLM assistance and runtime LLM inference are the same thing.

4. Answer

A, B. (C is wasteful/risky; D conflates writing reviewed code with live inference.) **Principle:** determinism for known structure, constrained, grounded LLM use only for genuinely unstructured meaning.

- A. From "agnostic by convention" to "agnostic by contract" (validation at the boundary).
- B. A malformed record fails loudly at the connector edge, not deep in a later audit.
- C. Pydantic emits JSON Schema that feeds MCP tool schemas and LLM structured output.
- D. It makes the code stdlib-only.

5. Answer

A, B, C. (D is backwards — it adds a dependency; that's the accepted trade for `src/`.)

Principle: push validation to the boundary; one schema definition serves many layers.

G. Process (SDD meta)

Q16 (select all that apply). Why turn decisions into ADRs and insights into backlog items *before* the code they affect?

- A. To keep a traceable, reviewable record of *why*.
- B. So learning-phase insights become tracked work instead of evaporating.
- C. To deliberately slow development down.
- D. Because intent is an artifact that should survive beyond a chat or people's memory.

6. Answer